

TRIO + Specmatic: Complete Spec-First Engineering Journey

A detailed engineering report documenting the adoption of Specmatic in the TRIO Agentic AI Assistant project, including contract testing, schema resiliency testing, API coverage, Ollama mocking, CI/CD automation, review feedback, debugging, and final results.

1. Background and Objective

TRIO is an Agentic AI Assistant built using FastAPI, Ollama, SQLite, Speech-to-Text and Text-to-Speech capabilities.

The objective of this project was to apply Specmatic's Spec-First Engineering principles to a real-world AI application.

Before integrating Specmatic into TRIO, the API documentation existed primarily as a development artifact. The goal was to transform the OpenAPI specification into a single source of truth that could drive implementation validation, mocking, contract testing, resiliency testing, coverage reporting and CI automation. The work involved creating executable contracts, improving API documentation, adding example pairs, debugging failing tests, building a mock of the Ollama API, integrating automated testing into GitHub Actions and ultimately achieving complete test success. The project demonstrated how specifications can become active engineering assets instead of passive documentation. Every improvement made during this effort was guided by actual test results and reviewer feedback, resulting in a significantly more robust API ecosystem.

2. Understanding the TRIO Architecture

TRIO consists of multiple interconnected subsystems. The FastAPI backend serves as the orchestration layer for all client interactions. A manager agent receives incoming requests and routes them to appropriate agents. SQLite stores conversation history and metadata. Ollama provides local language model capabilities for chat generation and reasoning. Speech-to-Text functionality converts user audio into text, while Text-to-Speech converts generated responses into audio.

The API exposes endpoints for health monitoring, agent discovery, system information, conversation management, chat generation and voice synthesis. Because these subsystems interact with each other, testing only individual components would not provide confidence in the overall system. Contract testing offered a way to validate complete end-to-end flows. The architecture made TRIO an ideal candidate for Specmatic because it contains both deterministic APIs and AI-assisted workflows that benefit from executable specifications and controlled mocking.

3. Learning Specmatic Before Applying It

Before modifying TRIO, the complete Specmatic learning curriculum was completed. This included hands-on exercises in contract testing, API mocking, schema resiliency testing, coverage analysis, backward compatibility testing, workflow testing using Arazzo, Kafka contract testing, MCP auto-testing and AI-assisted development workflows. Completing the course provided a strong understanding of how executable specifications can be used throughout the software development lifecycle. Rather than immediately applying changes to TRIO, the learning exercises were first used to understand the difference between documentation and executable contracts. This knowledge later proved valuable when diagnosing failures, understanding coverage reports and interpreting reviewer feedback. The course also highlighted best practices such as paired examples, contract-first design and the importance of mocking external dependencies.

4. Building the Initial OpenAPI Specification

The first major task was creating a comprehensive OpenAPI specification for TRIO. The specification documented request schemas, response schemas, validation rules, status codes and example payloads. Endpoints included health checks, agent listing, system information, conversation creation, conversation retrieval, title updates, chat generation and voice synthesis. Particular attention was given to defining response structures so that Specmatic could validate actual runtime responses against documented behavior. Initially the specification focused mainly on successful responses. This approach allowed happy-path testing to begin quickly but later revealed the importance of documenting validation and error responses as well. The specification became the foundation for all subsequent testing and automation work.

5. Initial Contract Testing Results

After the OpenAPI contract was created, Specmatic contract tests were executed against the running FastAPI backend.

The first execution validated that endpoints correctly handled valid requests. Happy-path scenarios passed successfully, confirming that the documented request and response structures matched actual implementation behavior. This was an important milestone because it established trust between the specification and the backend. However, these tests only verified success scenarios. They did not verify how the system behaved when receiving invalid input. As a result, additional testing techniques were required to evaluate robustness and error handling capabilities. This led directly to the next phase: schema resiliency testing.

6. Schema Resiliency Testing

Schema resiliency testing dramatically expanded test coverage. Instead of sending only valid requests, Specmatic automatically generated invalid variations by mutating strings into booleans, numbers and null values, omitting required fields and removing entire request bodies. These generated scenarios simulated real-world client mistakes. The objective was to verify that the backend rejected malformed requests gracefully with appropriate validation errors rather than crashing or returning unexpected responses. This testing mode provided much deeper insight into API robustness than traditional happy-path testing. It also revealed issues that would likely never have been discovered through manual testing alone.

7. First Major Failure Analysis

The first resiliency execution produced 39 tests with 32 successes and 7 failures. At first glance it appeared that the implementation contained seven defects. After detailed investigation, however, the failures were divided into two categories. Some failures were genuine specification gaps, while others represented implementation or validation issues. This distinction became one of the most valuable lessons of the project. Not every failed contract test indicates a code bug. Sometimes the backend behaves correctly but the specification does not accurately describe that behavior. Learning to identify the correct layer responsible for a failure significantly improved debugging efficiency and contract quality.

8. Fixing Missing 422 Validation Responses

Many failed tests were caused by undocumented validation responses. FastAPI correctly returned 422 Unprocessable Entity for invalid requests, but the OpenAPI specification only documented 200 success responses. Because the specification did not declare 422 responses, Specmatic could not validate them and therefore reported failures. The solution involved adding `ValidationErrorResponse` schemas and documenting 422 responses for conversation creation, title updates, chat generation and voice synthesis endpoints. Once these responses were formally included in the contract, Specmatic successfully validated the backend behavior and the associated failures disappeared. This improvement made the specification a more accurate representation of actual runtime behavior.

9. Reviewer Feedback and Example Pairing

During review, feedback highlighted that some APIs lacked properly paired request and response examples. Specmatic best practices recommend that every request example be matched with a corresponding response example using the same name. This enables executable example-driven testing. Additional examples were therefore created and aligned for health checks, agent listing, system information, conversations, chat

and voice endpoints. The improvement increased documentation quality while also improving test execution reliability. Each example became a concrete executable scenario capable of validating real application behavior. This change directly addressed reviewer concerns and strengthened the specification.

10. Moving From Hardcoded Responses to Ollama Mocking

One reviewer observation focused on the use of hardcoded responses during testing. While hardcoding enabled quick verification of chat endpoints, it was not considered a best practice because it bypassed actual workflow behavior.

Instead, a dedicated Specmatic mock of the Ollama API was created. A separate OpenAPI specification describing the subset of Ollama endpoints used by TRIO was introduced. During CI execution, Specmatic launches a mock Ollama server that returns deterministic responses. TRIO communicates with this mock exactly as it would with a real model provider. This approach tests the complete integration flow while maintaining deterministic behavior and removing dependence on local AI models. The resulting solution is significantly more realistic and maintainable than hardcoded responses.

11. Creating the Ollama Mock Specification

The Ollama mock specification models the endpoints required by TRIO, including `/api/chat`, `/api/generate` and `/api/tags`.

Component names were aligned with the official Ollama specification and paired request-response examples were added to support executable mocking. Documentation was also created explaining the relationship between the mock specification and the official Ollama contract. This clarification addressed reviewer concerns regarding naming consistency and intentional reductions. The mock remained a faithful subset while still being practical for automated testing. By documenting these decisions clearly, future maintainers can understand exactly why the mock exists and how it relates to the official API.

12. Debugging Voice Endpoint Failures

The most challenging issue involved the voice synthesis endpoint. The endpoint worked correctly on local machines but returned failures during GitHub Actions execution. Investigation revealed that CI runners lacked required audio-related dependencies used by the Text-to-Speech pipeline. As a result, audio generation failed and the endpoint returned 503 responses. Multiple debugging sessions were conducted to identify the exact cause. Additional Linux dependencies were installed and the fallback TTS implementation was improved to operate reliably in headless environments. Once these

changes were introduced, the endpoint began returning valid WAV responses during automated testing and coverage improved significantly.

13. GitHub Actions Automation

A complete CI/CD pipeline was built using GitHub Actions. The workflow checks out source code, installs Python dependencies, installs required audio libraries, launches the Specmatic Ollama mock, starts the FastAPI backend, executes contract tests and resiliency tests, generates HTML reports and uploads artifacts for inspection. This automation ensures that every code change is validated against the specification. Instead of manually executing tests, developers can rely on the pipeline to continuously verify API correctness. The workflow transformed the specification from a local development tool into a central quality gate for the entire project.

14. Final Results

After incorporating all improvements, the final execution produced 48 tests, 48 successes, 0 failures, 0 errors and 100% API coverage. Every eligible endpoint was covered. Chat workflows were validated using the Ollama mock. Voice synthesis worked successfully within CI environments. Validation errors were documented properly. Request-response example pairs were aligned. Coverage reports confirmed complete verification of all operations. The final result demonstrated the effectiveness of combining executable specifications, mocking, resiliency testing and CI automation within a single engineering workflow.

15. Key Learnings and Conclusion

The most important lesson from this project was understanding the distinction between specification problems and implementation problems. Contract testing revealed documentation gaps. Resiliency testing exposed robustness concerns. Mocking eliminated dependence on external AI systems while preserving end-to-end workflow validation. CI automation ensured continuous verification. Most importantly, the OpenAPI specification evolved into a true single source of truth for implementation, testing, coverage analysis and validation. The journey from 39 tests with failures to 48 passing tests and 100% coverage demonstrated how Spec-First Engineering can significantly improve API quality, reliability and maintainability in complex AI-powered applications.